Well-Architected Design and Generative AI on AWS

AWS foundations, volume 3 · Version 2026.1

> Sample foundation document written for testing a domain-expert RAG. AWS services, limits and model availability change often; confirm details in the official AWS documentation.

1. The six pillars of the Well-Architected Framework

Pillar	Focus	Example design principles
Operational excellence	Running and improving workloads	Infrastructure as code, small reversible changes, learn from failures
Security	Protecting data, systems and assets	Strong identity foundation, traceability, security at all layers, encrypt data
Reliability	Recovering from failure and meeting demand	Automatic recovery, test recovery procedures, scale horizontally
Performance efficiency	Using resources efficiently	Use managed and serverless services, experiment, go global in minutes
Cost optimisation	Avoiding unnecessary cost	Consumption model, measure efficiency, stop spending on undifferentiated work
Sustainability	Minimising environmental impact	Maximise utilisation, use efficient hardware and managed services

The AWS Well-Architected Tool lets teams review workloads against these pillars and track improvement plans. Specialised lenses exist, including one for generative AI.

2. Disaster recovery strategies

Strategy	RPO / RTO	Cost
Backup and restore	Hours	Lowest

Pilot light	Minutes to tens of minutes; core data replicated, compute off	Low
Warm standby	Minutes; a scaled-down copy runs in the second region	Medium
Multi-site active/ active	Near zero; both regions serve traffic	Highest

Multi-AZ designs protect against data centre failures and are the baseline for production. Multi-region designs protect against regional events and are justified only when the business impact requires them.

3. Amazon Bedrock

Amazon Bedrock is a fully managed service for building generative AI applications with foundation models from Amazon and other providers through one API, without managing infrastructure. Prompts and outputs are not used to train the base models, and data stays in the region you use.

Capability	**What it does**
Model access	Invoke text, chat, embedding and image models through the Converse and InvokeModel APIs
Knowledge Bases	Managed RAG: connect data sources such as S3, chunk and embed documents, store vectors, and retrieve with citations
Agents	Orchestrate multi-step tasks with action groups that call Lambda functions or APIs described with OpenAPI
Guardrails	Content filters, denied topics, word filters, sensitive information (PII) redaction and contextual grounding checks, applied to inputs and outputs
Model customisation	Fine-tuning and continued pre-training on your data, producing private model copies
Provisioned Throughput	Reserved capacity for predictable, high-volume use or custom models
Evaluation	Automatic and human evaluation of models and RAG pipelines

4. Building a RAG application on Bedrock

1. Put documents in an S3 bucket with encryption and least-privilege access.
2. Create a Knowledge Base, choose an embedding model and a vector store (for example OpenSearch Serverless or Aurora PostgreSQL with pgvector), and choose a chunking strategy.
3. Sync the data source; re-sync when documents change.
4. Query with the Retrieve API (passages only) or RetrieveAndGenerate (passages plus a generated answer with citations).
5. Attach a Guardrail with contextual grounding checks so ungrounded answers are blocked.
6. Use metadata filters to limit retrieval by tenant, team or document type.
7. Log invocations with model invocation logging and CloudWatch, and evaluate with a golden question set.

```
import boto3
client = boto3.client("bedrock-agent-runtime")
resp = client.retrieve_and_generate(
    input={"text": "What is our refund window?"},
    retrieveAndGenerateConfiguration={
        "type": "KNOWLEDGE_BASE",
        "knowledgeBaseConfiguration": {"knowledgeBaseId": KB_ID, "modelArn": MODI
print(resp["output"]["text"])
```

5. Amazon SageMaker AI

SageMaker AI covers the full machine-learning lifecycle for teams that train or host their own models: managed notebooks and studios, training jobs on CPU and GPU clusters, hyperparameter tuning, a model registry, real-time, serverless, asynchronous and batch inference endpoints, and SageMaker JumpStart for deploying open models with a few clicks. Choose Bedrock when a managed foundation model fits; choose SageMaker or EC2 when you need full control of the model and serving stack.

6. Security for generative AI on AWS

- Use VPC interface endpoints for Bedrock so traffic stays on the AWS network.
- Restrict model access with IAM policies on specific model ARNs, and use SCPs to block unapproved models or regions.

- Encrypt Knowledge Base data and agent sessions with customer managed KMS keys.
- Give agent action-group Lambda functions least-privilege roles, and require confirmation for sensitive actions.
- Enable model invocation logging to a protected bucket for audit.
- Apply Guardrails, but keep deterministic checks for high-impact actions.

7. Cost levers

Lever	Effect
Right-size and schedule	Turn off non-production resources outside working hours
Savings Plans and Spot	Large discounts for steady or interruptible compute
Storage lifecycle	Move cold data to cheaper S3 classes automatically
Model choice	Use the smallest model that meets quality targets; cache prompts; batch offline jobs
Visibility	Cost allocation tags, AWS Budgets alerts and Cost Explorer